URL: https://www.youtube.com/watch?v=RK11i58Kldw

Young Kings

Matthew — Group Leader: Led system architecture, design rationale, and dependency diagram.
Jason — Presenter 1: Wrote the abstract, introduction, overview, and system evolution; created and presented half of the slides.
Miguel — Presenter 2: Wrote the conclusions, lessons learned, limitations, and implications; created and presented half of the slides.
Stefan: Documented subsystem functionality and interactions; verified component responsibilities.
Andrew: Documented control/data flow, concurrency, and use-case sequence diagrams.
Jose: Prepared the AI collaboration report, data dictionary, naming conventions, and references; finalized and submitted the report.

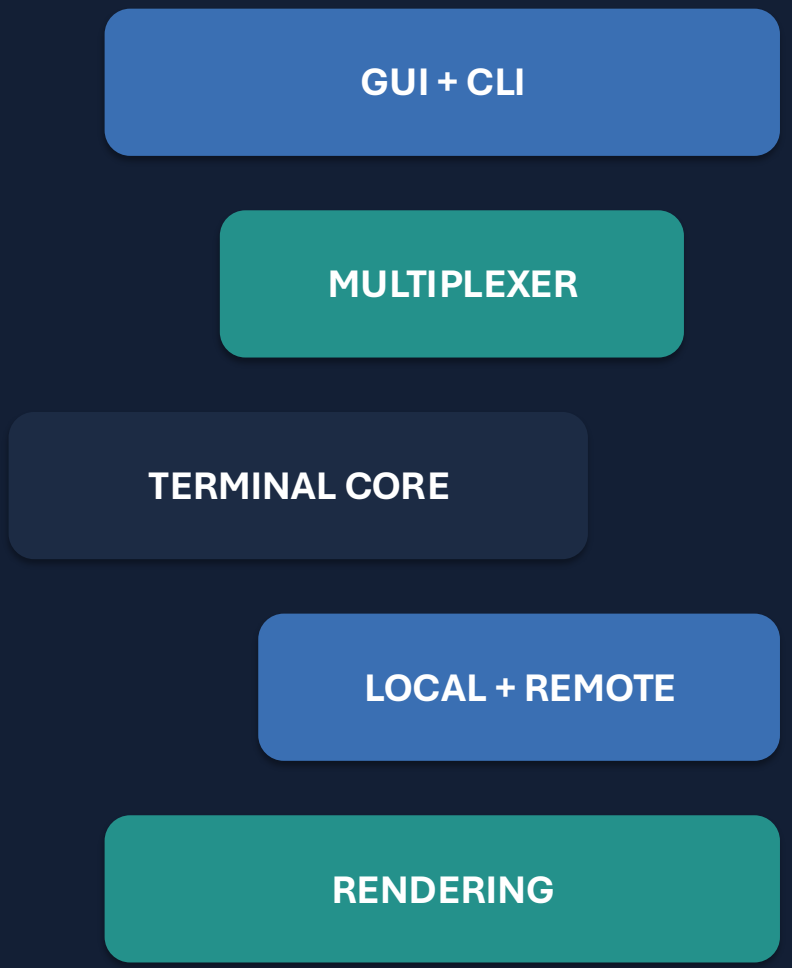
SOFTWARE ARCHITECTURE • CISC 322

Inside WezTerm

How a terminal emulator becomes a session manager

Conceptual architecture • 12 components • local and remote workflows

Group presentation



1. What does WezTerm do?

The architecture must support more than drawing text.

TERMINAL EMULATION
Interpret program output and maintain screen state

MULTIPLEXING
Organize panes, tabs, windows and workspaces

REMOTE ACCESS
Attach to sessions across machines

How we derived the model

- Started with user-facing documentation and features
- Mapped concepts to Rust packages and declared dependencies
- Grouped packages into 12 conceptual components
- Checked the model against two use cases and other terminals

KEY METHOD

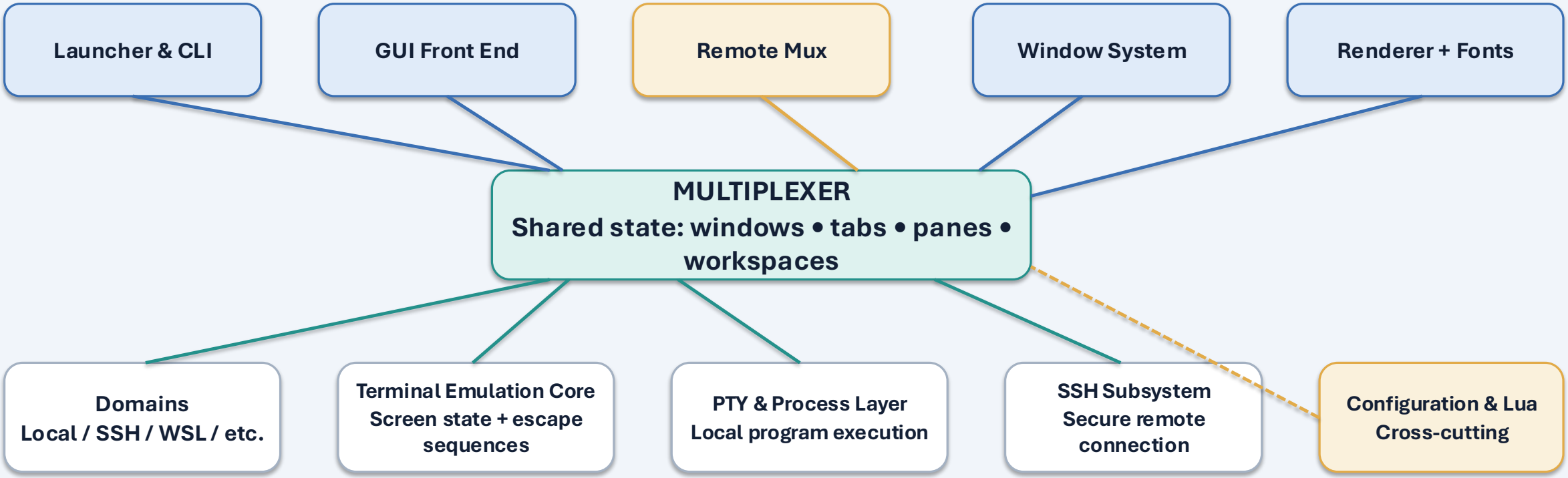
Document features → inspect repository → group responsibilities → trace interactions → validate with use cases

The component boundaries are our interpretation, not an official architecture.

Conceptual model derived from WezTerm documentation and repository structure

2. Conceptual architecture: 12 components

The Multiplexer is the central hub; configuration cuts across the system.



Legend: solid arrows = primary interaction/dependency; dashed gold arrow = cross-cutting configuration influence.

Simplified view: some relationships are omitted for readability. Renderer and Font Subsystem are shown together here.

Conceptual model derived from WezTerm documentation and repository structure

3. Why this architectural style?

One style alone does not explain both local and remote behaviour.

PRIMARY
Repository + change notification
Mux owns shared session state

REMOTE
Client-server
Remote Mux connects clients and servers

SUPPORTING
Relaxed layers + events + interpreter
Lua config and event callbacks

Why components interact this way

- GUI receives Mux notifications, then pulls visible pane content
- Terminal Core owns screen state; Renderer only draws it
- Front ends do not access PTYs directly
- Remote panes use the same Mux-facing interface as local panes

Tradeoff

BENEFIT
Shared session model simplifies local/remote integration.

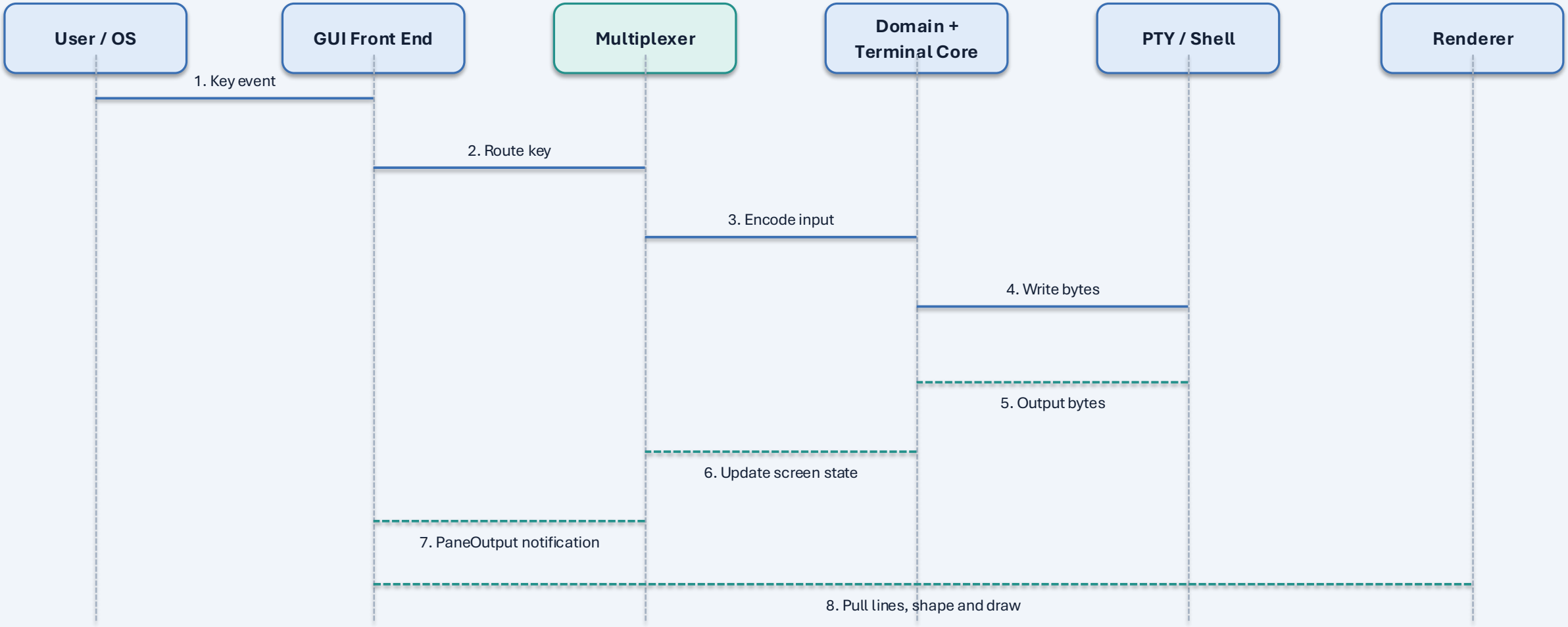
COST
More components and a protocol to coordinate.

This is a conceptual classification based on evidence, not WezTerm's official label.

Conceptual model derived from WezTerm documentation and repository structure

4. Sequence: running a local command

Control flows toward the shell; output returns toward the screen.

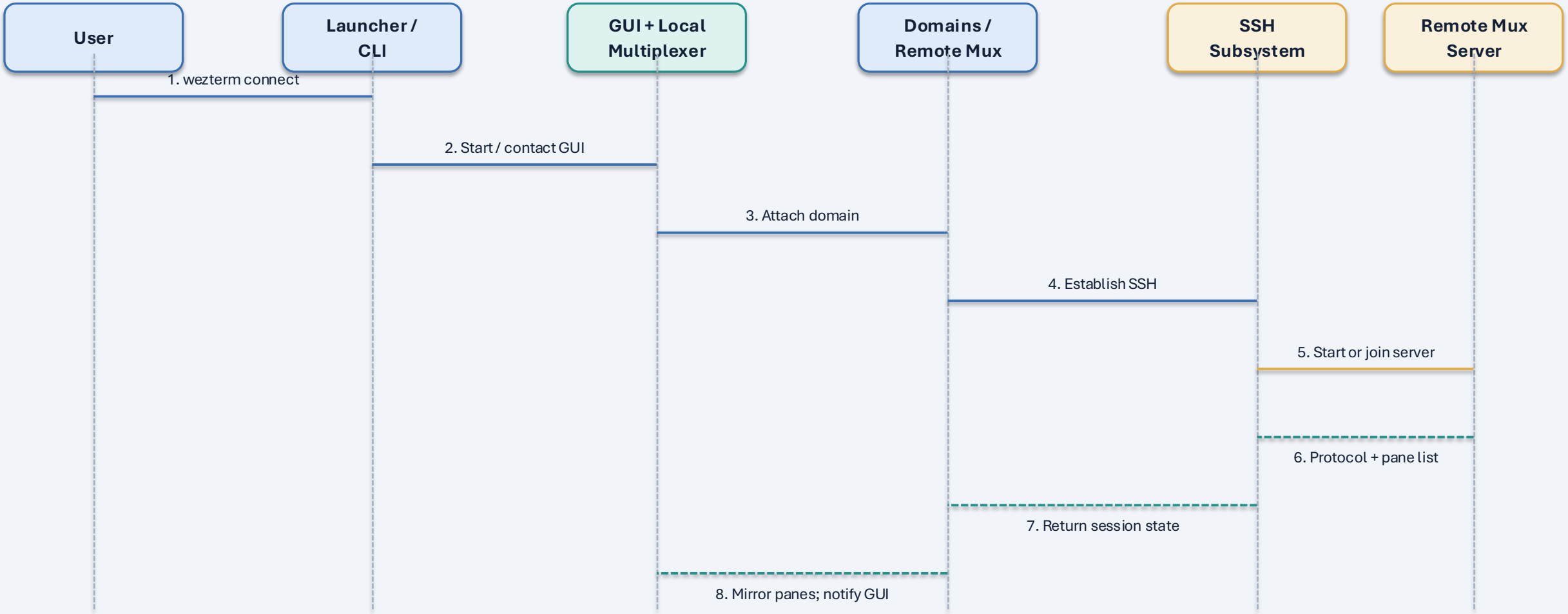


Legend: solid blue = control/input request; dashed teal = output, notification or render response. Sequence is conceptual and omits internal calls.

Conceptual model derived from WezTerm documentation and repository structure

5. Sequence: attaching to a remote session

The local GUI uses the same Multiplexer model after the remote connection is established.



Legend: solid = request/connection; dashed = response/state updates. SSH is the transport; Remote Mux coordinates session pro tocol.

Conceptual model derived from WezTerm documentation and repository structure

6. Concurrency and division of responsibility

Concurrency supports responsiveness and keeps session ownership clear.

PROCESSES
GUI • headless mux server • CLI
Communicate over sockets / remote transports

THREADS
Per-pane reader + parser
Separate blocking reads from parsing

ASYNC TASKS
Mux client connections and config file watching

Responsibility boundaries

- Terminal Core: parse escape sequences and own screen state
- Multiplexer: own sessions and broadcast state-change notifications
- GUI / Renderer: handle interaction and display, not PTY management
- Domains + process layer: connect panes to local or remote programs
- Configuration & Lua: customize behaviour across components

Implication: teams can reason about components separately, but must coordinate interfaces and shared protocols.

Conceptual model derived from WezTerm documentation and repository structure

7. Alternatives, limitations, lessons

The architecture is a defensible model, not the only possible model.

Alternatives considered

Strict layered model
Rejected: GUI and session layer are siblings in important interactions.

Terminal-only model
Rejected: misses the integrated Multiplexer and remote sessions.

Separate external multiplexer
Contrast: tmux is standalone; WezTerm integrates mux behaviour with a GUI.

Limitations and lessons

- No official architecture document; package dependencies do not prove every runtime interaction
- Some concurrency details come from source inspection, not user documentation
- Use cases exposed missing arrows and clarified component boundaries
- A future revision should trace critical calls and validate diagrams with maintainers

TAKEAWAY
The Multiplexer is the key architectural decision connecting local UI, session state and remote access.

Questions / feedback: Are the component boundaries useful, and where would you model them differently?

Conceptual model derived from WezTerm documentation and repository structure